



Large-Scale Computing

Data replication, CAP theorem, CRDTs

Bartosz Baliś

Data replication



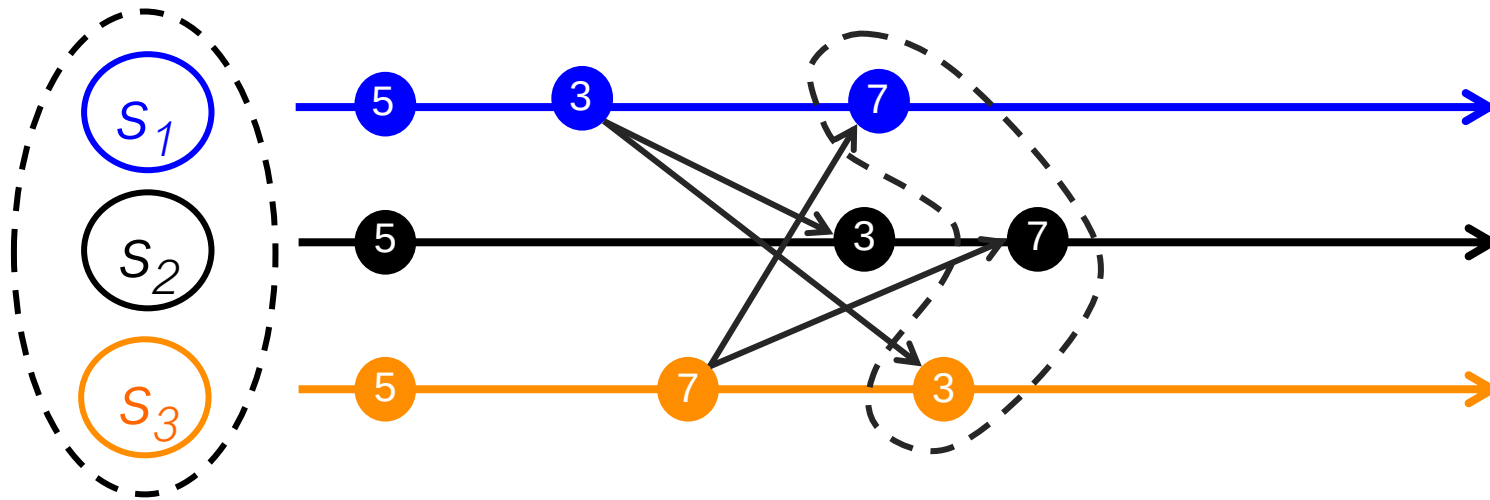
- » Essential in large-scale scalable distributed systems
- » **Data replication** = copies of the same data (system state) are kept on multiple locations (nodes/machines)

Benefits of data replication

- » **Improved durability** (lower risk of data loss)
- » **Higher availability** (failover to another replica)
- » **Increased scalability** (system load can be spread across replicas)
- » **Reduced latency** (requests can be done on replicas close to users)

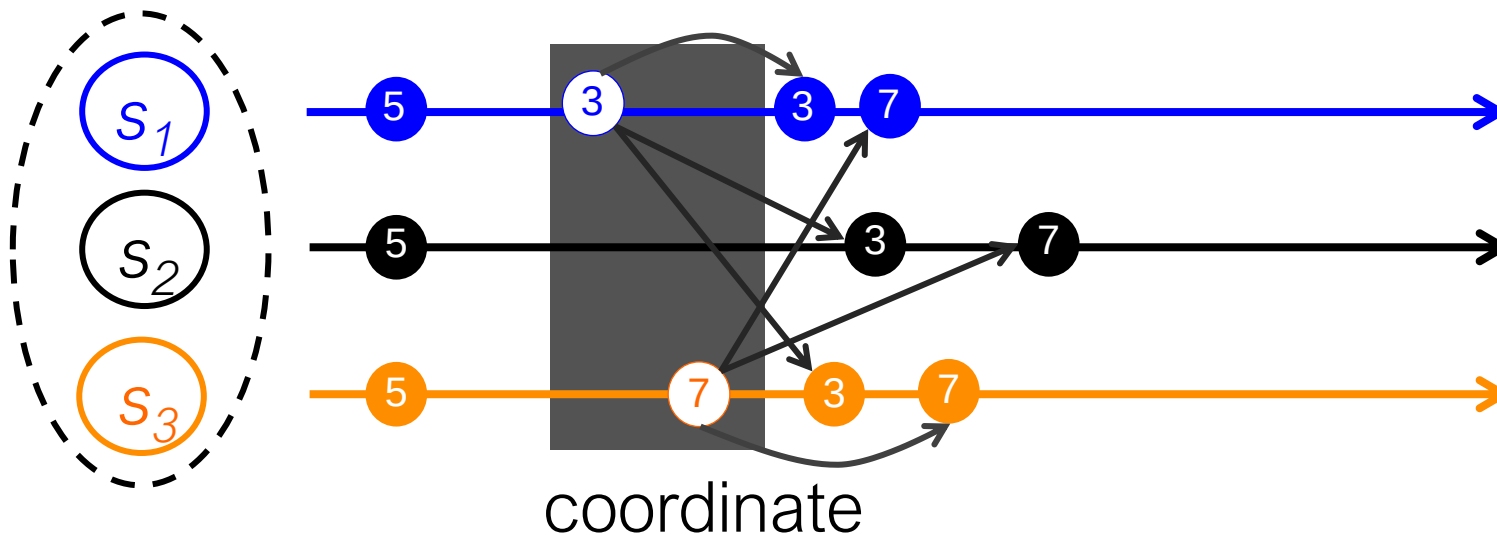
Challenge: consistency

- » Concurrent updating replicas may lead to different results → inconsistent data
- » How to ensure strong consistency?



Strong consistency

- » All replicas execute updates in same total order
 - Deterministic updates: **same update on same objects**
→ **same result**



Strong consistency

- » All replicas execute updates in same total order
 - Deterministic updates: same update on same objects
→ same result
- » Requires coordination and consensus to decide on total order of operations
 - N-way agreement, basically serialize updates → very expensive!

Limits of scalability

Two factors limiting scalability:

- » **Contention**: waiting for access to shared resources
- » **Coherency**: delay for data to become consistent (communication between processes)

Gunter's Universal Scalability Law (for synchronous systems)

$$C(N) = \frac{N}{1 + \alpha(N - 1) + \beta N(N - 1)}$$

Relative capacity (throughput) $\rightarrow C(N)$

Number of processing units (processors / nodes) $\rightarrow N$

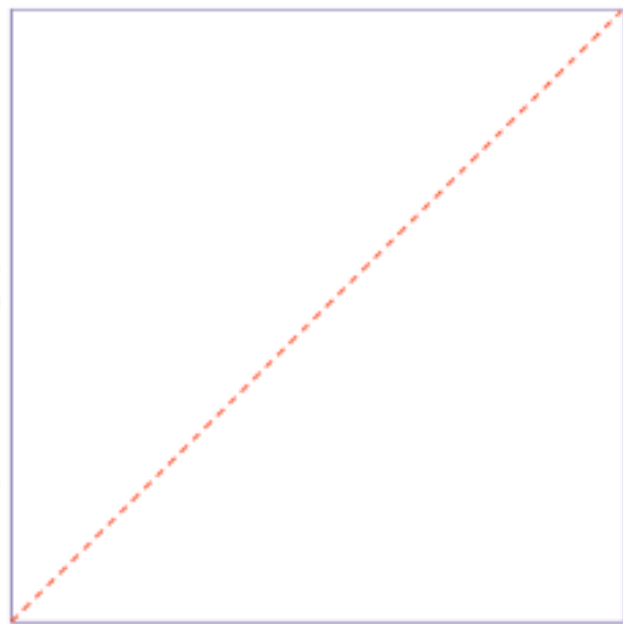
Contention overhead $\rightarrow \alpha(N - 1)$

Coherency overhead $\rightarrow \beta N(N - 1)$

$\beta=0$

Amdahl's

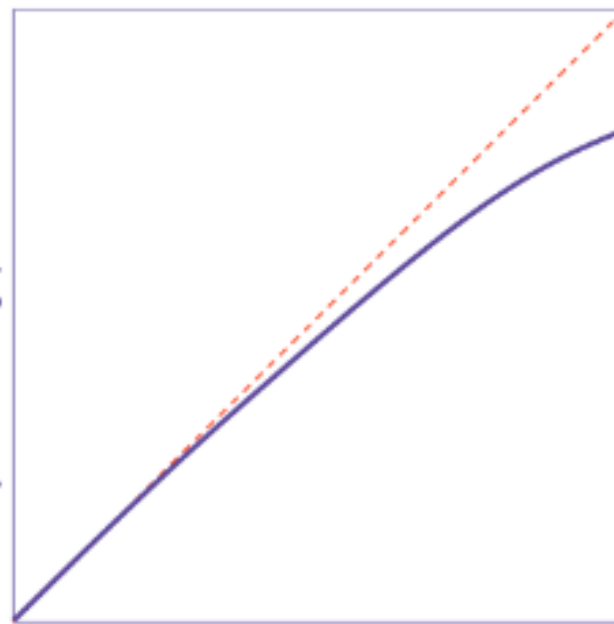
System throughput



Load on the system

$$\alpha = 0, \beta = 0$$

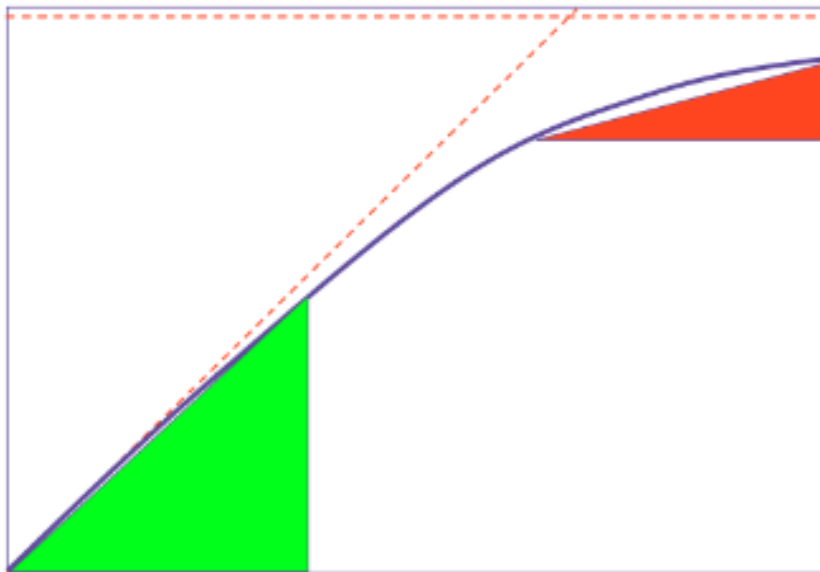
System throughput



Load on the system

$$\alpha > 0, \beta = 0$$

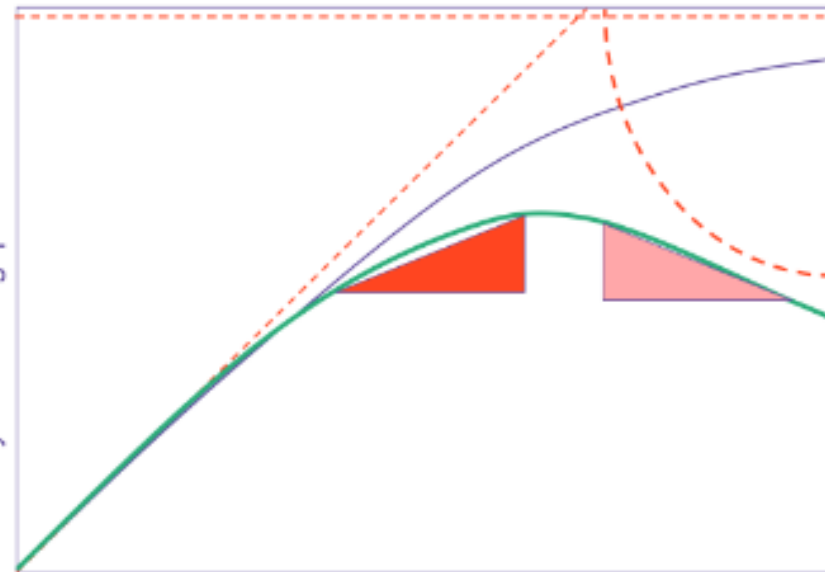
System throughput



Load on the system

$$\alpha \gg 0, \beta = 0$$

System throughput



Load on the system

$$\alpha \gg 0, \beta > 0$$

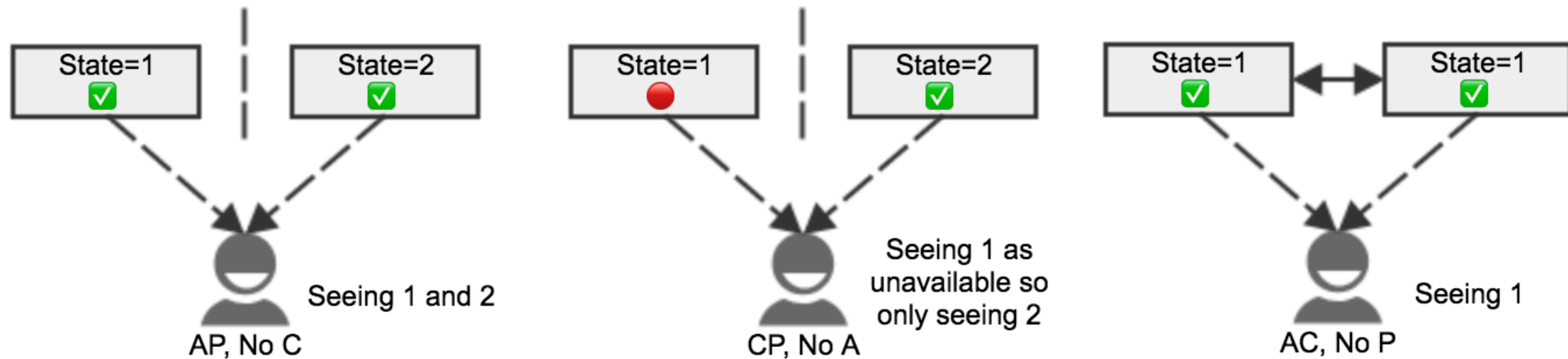
CAP theorem

Any distributed shared-data system can have at most two of three desirable properties:

- » **Consistency**: all nodes see the same data at the same time – every read returns the most recent write (or an error)
- » **Availability**: every request receives response (but the data may be outdated)
- » **Partition tolerance**: the system continues to function even when network splits occur (some nodes cannot communicate with others)

CAP theorem

Simplest case: 2 nodes



CAP theorem

- » CAP led to new generation of distributed systems
- » NoSQL movement: availability first, consistency second
- » Traditional ACID – the opposite

CAP – 12 years later

- » In practice “2 of 3” is misleading
- » For ‘wide area’ distributed systems partitions are inevitable ...
- » However:
 - ‘A’ in CAP stands for ‘100% availability’
 - You need to forfeit ‘C’ or ‘A’ during actual partition
 - Partitions are rare

CAP – 12 years later

Choice between 'C' and 'A' ...

- » can occur many times within the same system
- » and at very fine granularity

Which means that:

- » subsystems can make different choices
- » these choices depend on circumstances, even specific data or user involved

CAP – 12 years later

- » All three properties are continuous rather than binary choices
- » Consequently, systems should:
 - allow perfect 'C' and 'A' most of the time
 - apply partition recovery strategy

CAP – 12 years later

Partition recovery strategy has three steps:

- » **Detect** partitions
- » **Enter** an explicit **partition mode** (limit some operations)
- » **Initiate a recovery process** to restore consistency and compensate for mistakes made during the partition

CAP and latency

- » In classic interpretation CAP ignores latency
- » In practice, latency and partitions are strongly related
- » The ***partition decision*** must be taken by a system during a **timeout**:
 - Cancel the operation (decrease 'A')
 - Proceed with the operation (risk non-'C')

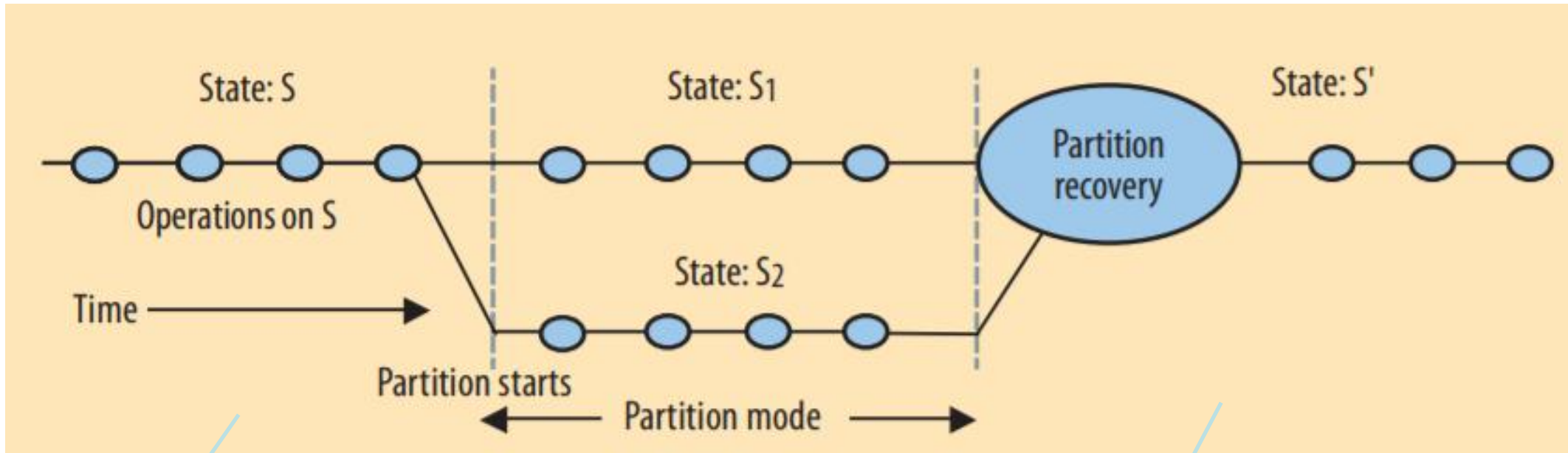
Detecting partitions

Pragmatic approach: partition = failing to achieve consistency within a time bound

- » Time bound can be set according to target response times
- » Tighter bounds → system enters partition mode more often (even when network is just slow)

No global notion of a partition – some nodes may detect it, some not

Managing partitions



State consistent until partition

Both sides enter partition mode, create two concurrent states S1 and S2

Recovery: system merges S1 and S2 into consistent S'
Also compensates for mistakes

One-sided partition

One side enters the partition mode, the other side responds normally

Examples:

- » systems with quorum – one side will have a quorum and continue operation
- » systems with ‘offline mode’ (also known as ‘disconnected operation’) – common in web applications whose part runs on the client with client-side storage

Partition recovery

Two strategies:

- » Strategy 1: limit operations (decrease '**A**')
- » Strategy 2: record extra information to help during recovery

Partition recovery

- » Which operations should proceed?
 - Depends on the invariants the system must maintain
- » Example invariant: 'keys in a table are unique'
 - During partition duplicate keys may be allowed which is easy to discover during recovery
- » Charging a credit card cannot be performed during a partition
 - Strategy: record it and execute after recovery

Partition recovery

- » Two problems must be solved:
 - State on both sides must be consistent
 - Mistakes made during the partition must be compensated

Reaching consistent state

- » Roll back to a last known consistent state and repeat all operations from both sides of partitions in a certain deterministic order
- » Causal ordering (vector clocks) helps deducing which operations cause inconsistencies
- » Limit allowable set of operations during partition to enable automated merge

Example: concurrent document editing

- » Changes in document are applied in local copies (in memory of browsers)
- » Then changes are asynchronously propagated through a server
- » E.g. Google uses algorithm based on Operational Transformation
 - requires a central server

Commutative operations

If all operations are commutative automated merge is always possible

- » **Merge operation logs** (from different partitions)
- » **Order them** in some way
- » **Execute them** (on both sides of the partition)

Achieving commutativity is hard

Compensating for mistakes

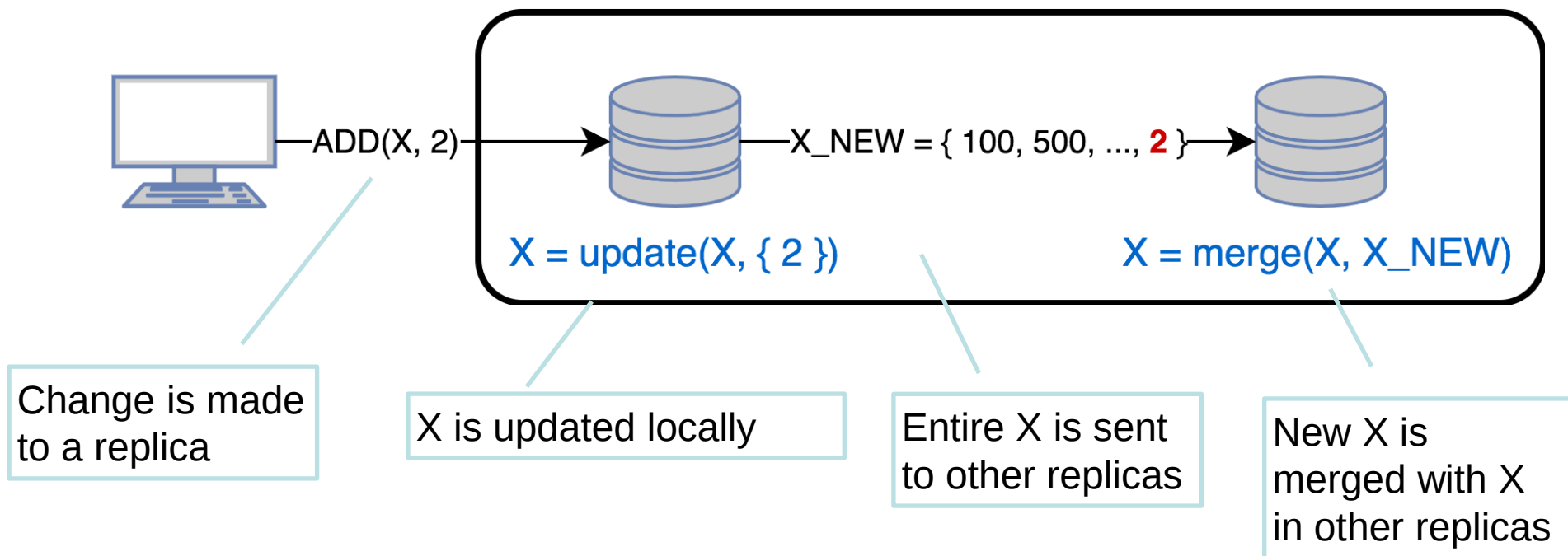
- » **Fix invariants** violated during the partitioning
- » Trivial strategy: “last writer wins”
- » Smarter strategy: merge operations
- » Human escalation, e.g. overbooking, order placed twice (externalized mistake) – customer service needs to compensate
- » SAGA transactions (no locks, compensating actions)

CRDTs

- » Conflict-free Replicated Data Types
- » Class of data structures that provably converge after a partition
- » Concurrent updates do not require synchronization
- » Enable **strong eventual consistency (SEC)**

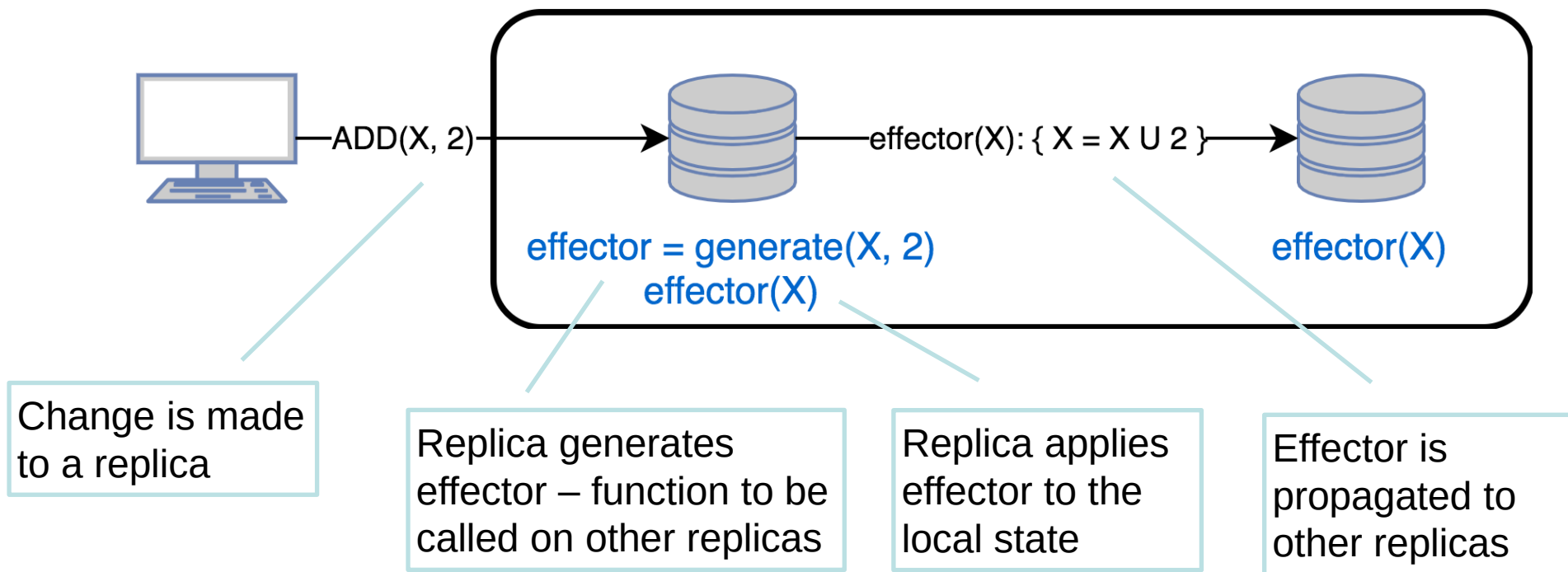
Types of CRDTs: **state-based** (CvRDTs – **convergent** RDTs)

- » Replicas propagate changes by sending a full state of the objects
- » A `merge()` function must be defined to merge incoming changes with the current state



Types of CRDTs: **operation-based** (CmRDTs – Commutative RDTs)

Replica propagates changes by sending the operation to all replicas



CRDTs: algebra (state-based)

$x, y, z \in S$ *states* $\cdot$ *merge operation*

- » **Idempotent function:** $x \cdot x = x$
- » **Commutative function:** $x \cdot y = y \cdot x$
- » **Associative:** $(x \cdot y) \cdot z = x \cdot (y \cdot z)$
- » **Partial order:** reflexivity + transitivity + antisymmetry
- » **Semilattice:** $(S, \cdot)$ form *semilattice*, i.e. partially ordered set in which every (non-empty) subset has a least upper bound (LUB)
- » **Monotonicity:** state must increase monotonically

CRDT Theorem (state based)

Definition (Monotonic semilattice object)

A state-based object, equipped with partial order $\leq$, noted $(S, \leq, s_0, q, u, m)$, that has the following properties, is called a monotonic semi-lattice:

- (i) Set S of payload values forms a semilattice ordered by $\leq$.
- (ii) Merging state s with remote state s' computes the LUB of the two states, i.e., $s \bullet m(s') = s \sqcup s'$
- (iii) State is monotonically non-decreasing across updates, i.e., $s \leq s \bullet u$.

Theorem (Convergent Replicated Data Type). Assuming eventual delivery and termination, any state-based object that satisfies the monotonic semilattice property is **Strongly Eventually Consistent**.

Operation-based CRDTs: requirements for SEC

- » **Reliable delivery** – all operations must be (eventually) delivered to all replicas
- » **Causal ordering and commutativity** of operations
 - If `effector()` is delivered in causal order then concurrent `effector()` have to commute **OR**
 - If `effector()` is delivered without respecting causal order then all `effector()` have to commute
- » **Idempotency**: operations must be idempotent (if they can be delivered more than once) or deduplicated

CRDT example:

G-counter (increment-only)

- » Usage: e.g. hits or likes counter
- » Vector V with size = amount of replicas

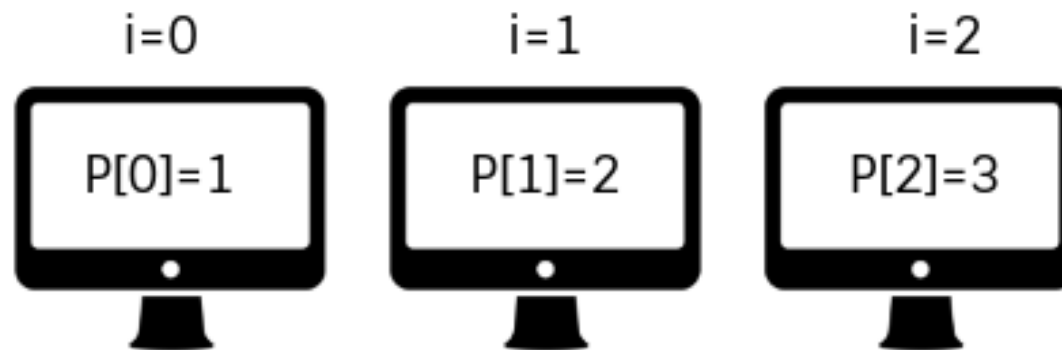
$$inc() : V[id()] = V[id()] + 1$$

$$value() : \sum_{i=0}^n V[i]$$

$$merge(C_1, C_2) : i \in [1..n] \text{ Result}[i] = \max(C_1.V[i], C_2.V[i])$$

- » PN-counter (inc-dec) uses two vectors: for increments and decrements

G-Counter: example



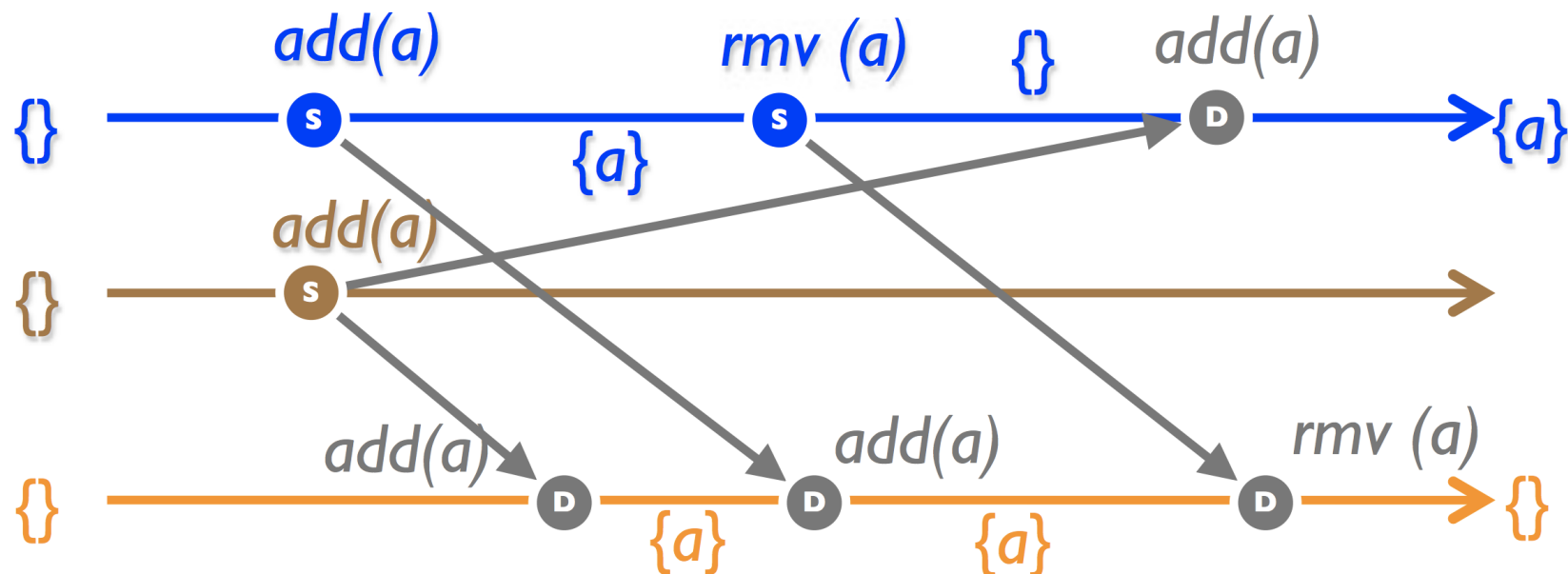
Before Merge at node $i=0$, $P=\{1,0,0\}$
at node $i=1$, $P=\{0,2,0\}$
and at node $i=2$, $P=\{0,0,3\}$
After Merge at all node $P=\{1,2,3\}$
and values of $X=2+(1+2+3)=8$

G-Counter algebra: lattice structure

- » **Elements:** Each state of the counter can be represented as a vector
 - For a system with three nodes (A, B, and C), the state could be represented as **[A: 1, B: 2, C: 0]**, meaning node A has incremented the counter once, node B twice, and C not at all
- » **Partial Order:** The lattice structure requires a partial order. Here, one state is "greater" than another if it is equal or greater in every element.
 - **[A: 2, B: 2, C: 0]** is greater than **[A: 1, B: 2, C: 0]**
- » **Least Upper Bound (LUB):** For any two states, the LUB is the element-wise maximum of these states.
 - For **[A: 1, B: 2, C: 0]** and **[A: 2, B: 1, C: 3]**, the LUB is **[A: 2, B: 2, C: 3]**
- » **Monotonicity:** The counter can only move in one direction which guarantees that the system's state will not regress

CRDT: Set

- » Set has two non-commutative operations: **add()** and **rmv()**
- » Naive implementation: **add()** and **rmv()** are executed sequentially as they arrive



G-Set (grow only set)

- » Simplest solution
- » We only allow adding elements to the set, removing elements is not permitted

2P-Set (2-phase set)

Two G-Sets (grow-only sets):

- » **A**: element set
- » **R**: tombstone set (tracks removed elements)
- » Removed element cannot be re-added

$$\text{lookup}(e) : e \in A \wedge e \notin R$$

$$\text{add}(e) : A = A \cup \{e\}$$

$$\text{rmv}(e) : R = R \cup \{e\}$$

$$\text{merge}(S_1, S_2) :$$

$$\text{Result. } A = S_1.A \cup S_2.A$$

$$\text{Result. } R = S_1.R \cup S_2.R$$

LWW-Element-Set

(Last-Writer-Wins-Element-Set)

- » addSet A, removeSet R
- » Each element in these sets has a timestamp
- » **add(e)** – adds element to set A
- » **remove(e)** – adds element to set R
- » **lookup(e)** – true if $(e \in A \wedge e \notin R) \vee (e \in A \wedge e \in R \wedge A.e.timestamp > R.e.timestamp)$

LWW-Element-Set

(Last-Writer-Wins-Element-Set)

merge (S, T):

$U.A = (S.A \cup T.A) \wedge (\forall e \in S.A, T.A \Rightarrow$

$U.A.e.timestamp =$

$\max(S.A.e.timestamp, T.A.e.timestamp))$

$U.R = (S.R \cup T.R) \wedge (\forall e \in S.R, T.R \Rightarrow$

$U.R.e.timestamp =$

$\max(S.R.e.timestamp, T.R.e.timestamp)$

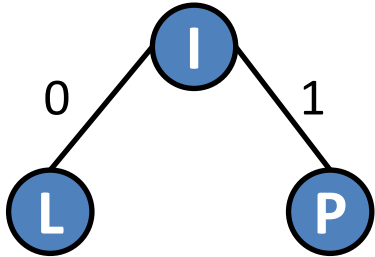
return U

Treedoc

- » CRDT for collaborative text editing
- » Text is a sequence of “atoms” (characters, inserted images, etc.)
- » Atoms are assigned unique, ordered Ids
- » Id space must be **dense** – for any two Id_1 and Id_2 , there always exists Id_3 :
 $Id_1 < Id_3 < Id_2$
- » Solutions: Ids are paths in a binary tree

Treedoc – example

replica₁



Total order "<":
infix traversal



- **Replicated representation:**

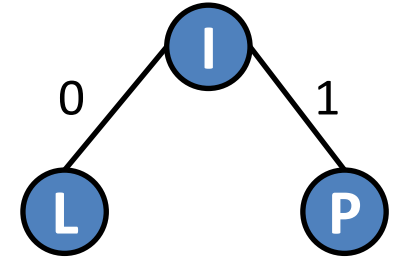
- Grow-only binary tree
- Stable, unique position ids

P = *1*

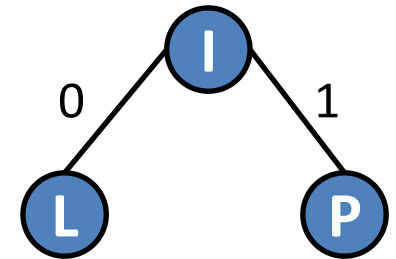
- **Sequence of atoms:**

- Ops: *read, addAt, removeAt*

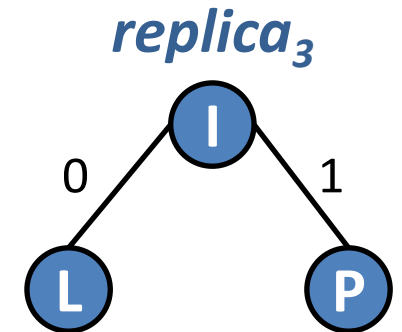
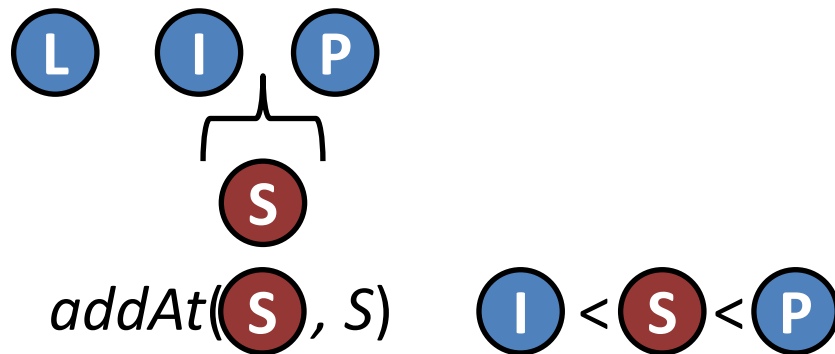
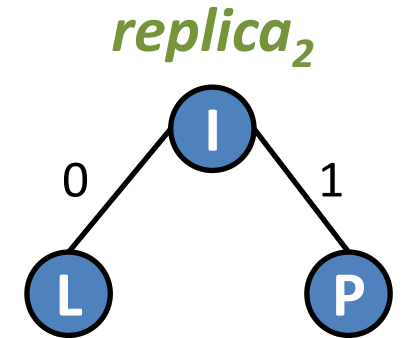
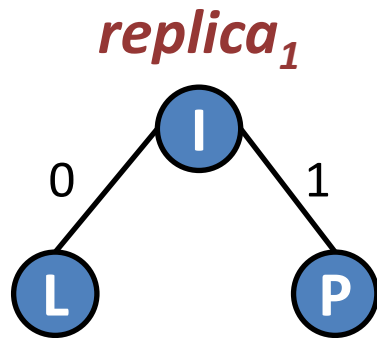
replica₂



replica₃

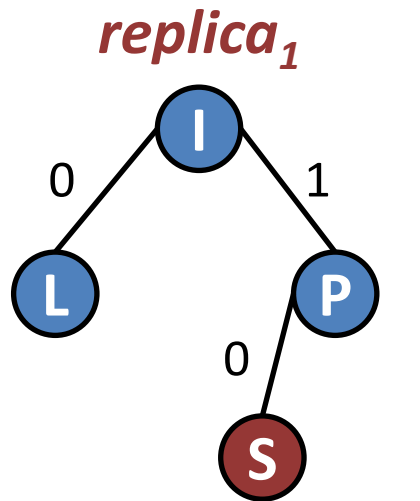


Treedoc – example



[Shapiro, Preguiça et. al, 2007, 2009]

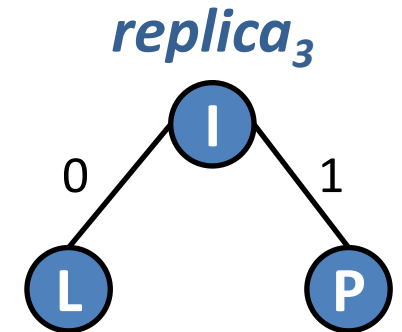
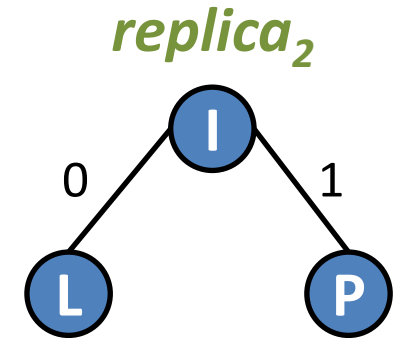
Treedoc – example



addAt((S), S)

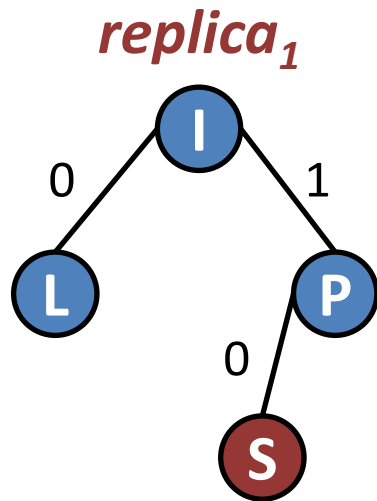


(S) = 10



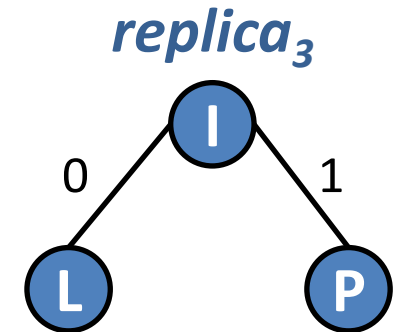
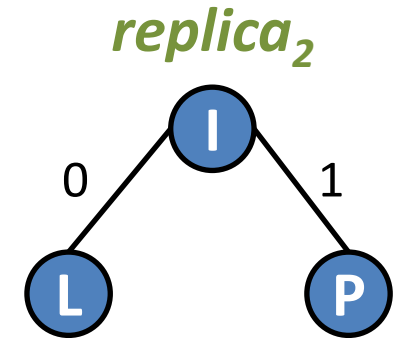
[Shapiro, Preguiça et. al, 2007, 2009]

Treedoc – example



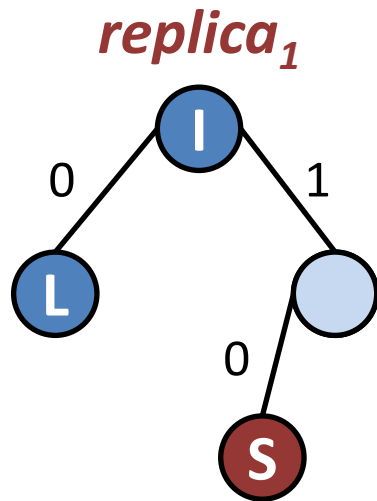
addAt((S), S)

removeAt((P))



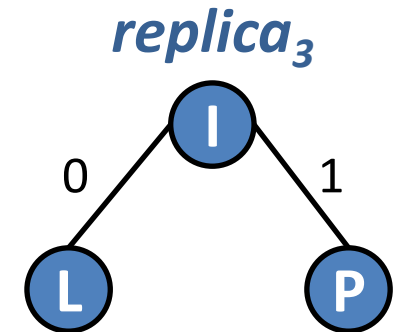
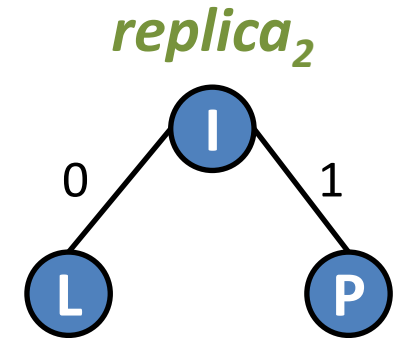
[Shapiro, Preguiça et. al, 2007, 2009]

Treedoc – example



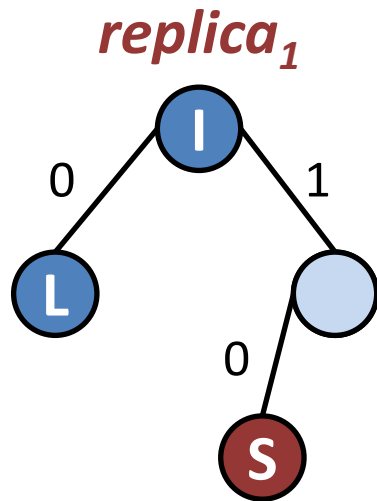
addAt((**S**), *S*)

removeAt((**P**))



[Shapiro, Preguiça et. al, 2007, 2009]

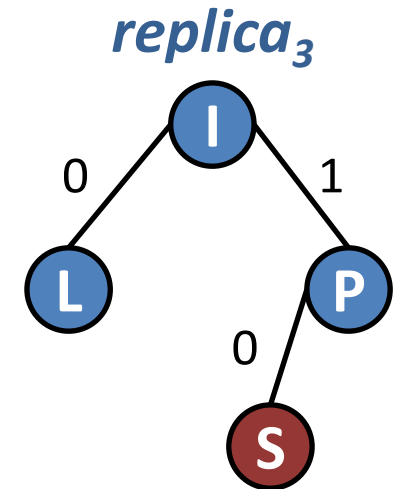
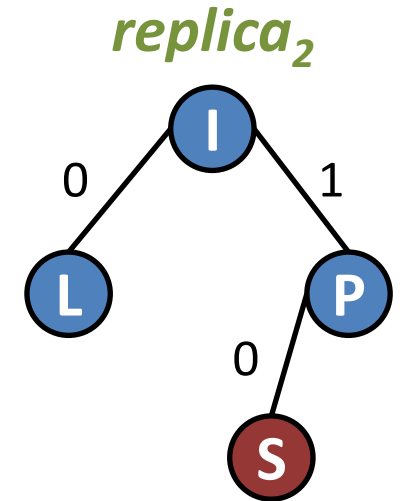
Treedoc – example



addAt(**S**, S)

removeAt(**P**)

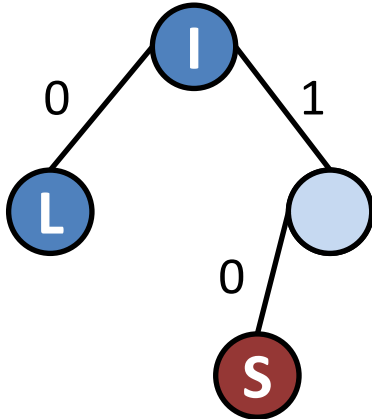
- **Operation-based replication:**
 - Immediate local execution
 - Propagate & replay



[Shapiro, Preguiça et. al, 2007, 2009]

Treedoc – example

replica₁

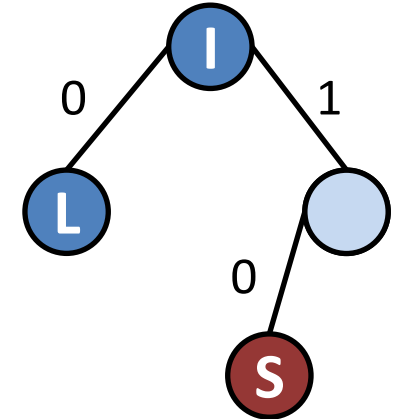


addAt((S), S)

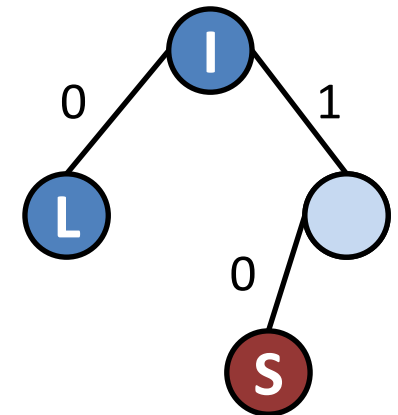
removeAt((P))

- **Operation-based replication:**
 - Immediate local execution
 - Propagate & replay

replica₂

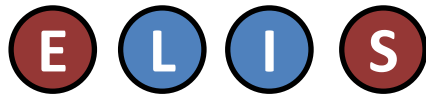
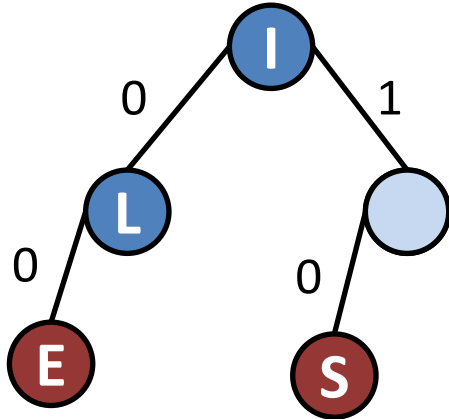


replica₃



Treedoc – example

replica₁

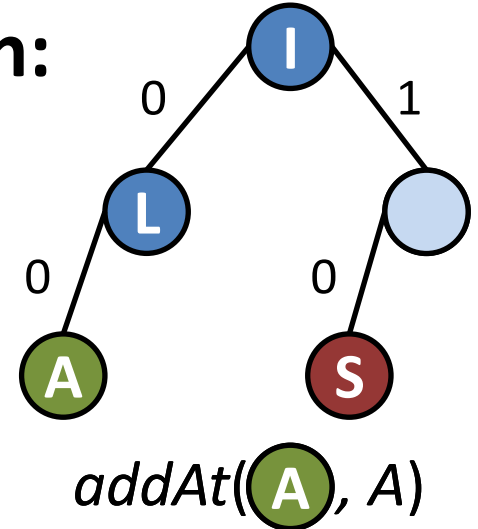


addAt(**E**, E)

?

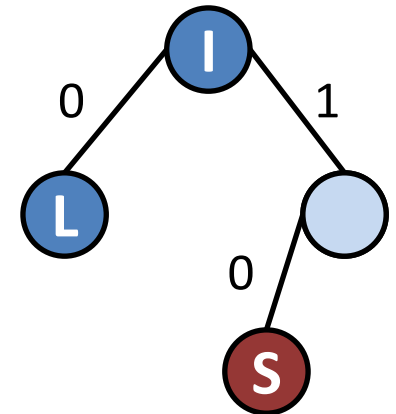
- **Operation-based replication:**
 - Immediate local execution
 - Propagate & replay

replica₂



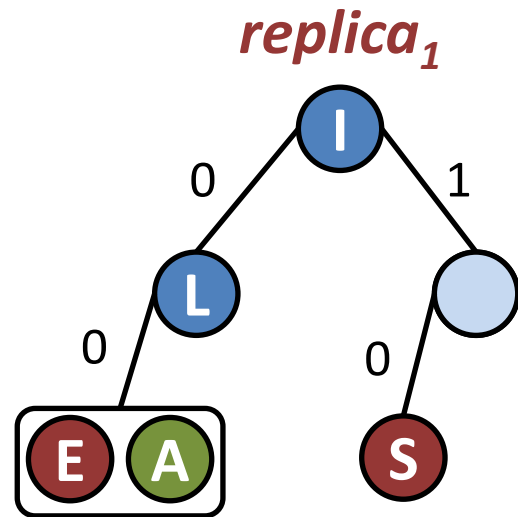
addAt(**A**, A)

replica₃



[Shapiro, Preguiça et. al, 2007, 2009]

Treedoc – example



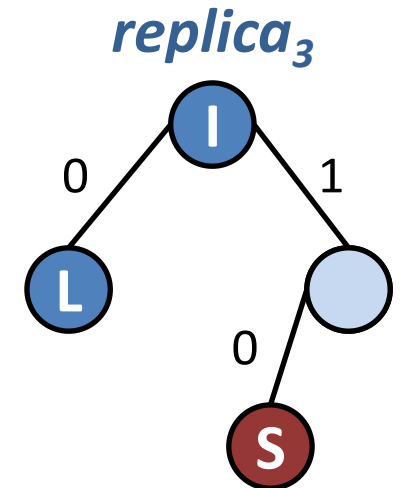
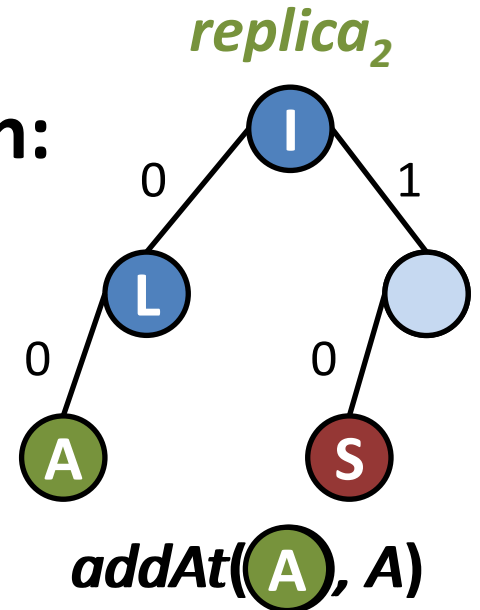
$addAt(\text{E}, E)$

$addAt(\text{A}, A)$

Predefined order:

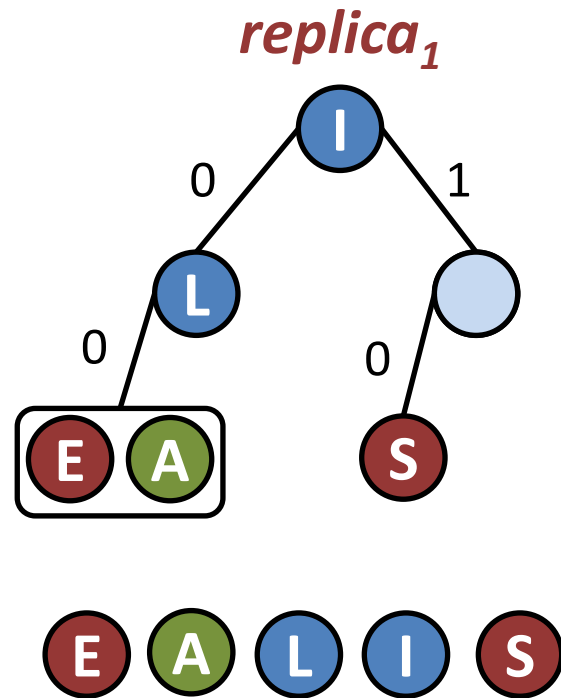
red < green < blue...

- **Operation-based replication:**
 - Immediate local execution
 - Propagate & replay



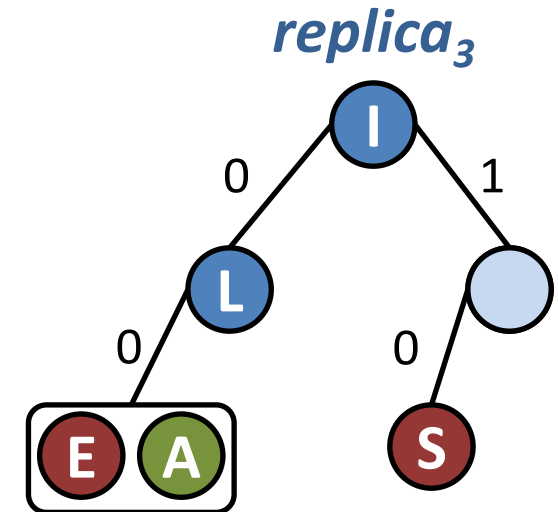
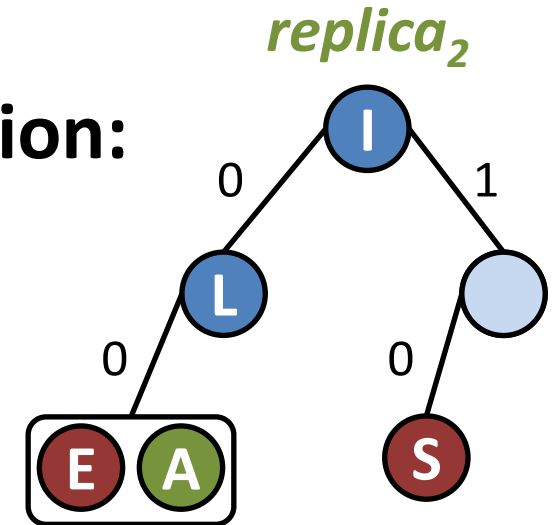
[Shapiro, Preguiça et. al, 2007, 2009]

Treedoc – example



- **Operation-based replication:**

- Immediate local execution
- Propagate & replay
- Concurrent commute
- Eventually consistent



Predefined order:

red < green < blue...

[Shapiro, Preguiça et. al, 2007, 2009]

CRDTs are now widely used...

(copied from Wikipedia)

- [Redis](#) is a distributed, highly available and scalable in-memory database that uses CRDTs for implementing globally distributed databases based on and fully compatible with Redis open source.
- [Riak](#) is a distributed NoSQL key-value data store based on CRDTs.
- [League of Legends](#) uses the Riak CRDT implementation for its in-game chat system, which handles 7.5 million concurrent users and 11,000 messages per second.
- [TomTom](#) employs CRDTs to synchronize navigation data between the devices of a user.
- [Phoenix \(web framework\)](#), a web framework written in [Elixir](#), uses CRDTs to support real time multi-node information sharing in version 1.2.
- [Facebook](#) implements CRDTs in their Apollo low-latency "consistency at scale" database.
- Teletype for [Atom](#) employs CRDTs to enable developers share their workspace with team members and collaborate on code in real time.
- Microsoft's [Cosmos DB](#) uses CRDTs to enable a multi-master write mode.
- [Apple](#) implements CRDTs in the Notes app for syncing offline edits between multiple devices.^[33]